Akademia Górniczo-Hutnicza im. Stanisława Staszica w Krakowie
WYDZIAŁ FIZYKI I INFORMATYKI STOSOWANEJ

KATEDRA ODDZIAŁYWAŃ I DETEKCJI CZĄSTEK

Projekt dyplomowy

Portable Identification Code Reader with 1-Wire Interface

Przenośny czytnik kodów identyfikacyjnych z interfejsem 1-Wire

Autor: Franciszek Urbański
Kierunek studiów: Informatyka Stosowana
Opiekun pracy: dr inż. Krzysztof Świentek

Kraków, 2026

Contents

Introduction

LUXE (Laser Und XFEL Experiment) [1] is an international experiment on the European XFEL (X-Ray Free-Electron Laser) in Hamburg, Germany, designed to investigate quantum electrodynamics phenomena in a very strong electromagnetic field. Collision of a high-power laser beam (350 TW) with a high-energy electron beam (16.5 GeV) will generate an electric field strong enough to cause spontaneous creation of electron-positron pairs in a vacuum, allowing researchers to study this phenomenon.

ECAL-P [1] is a high-density electromagnetic calorimeter in the LUXE experiment, built to measure positron's energy and spectra. It consists of 21 tungsten absorber plates, separated by silicon sensors (see figure 1). Such layered arrangement requires tens of homogeneous frontend electronics boards, all which are tested individually before assembly. Their characteristics are stored in a database and are updated after the detector is assembled and more tests are performed. Furthermore, during detector operation the behavior of each component is tracked, to understand parameter drift and other issues appearing due to detector aging. It is therefore very important to be able to reliably distinguish the boards from one another.

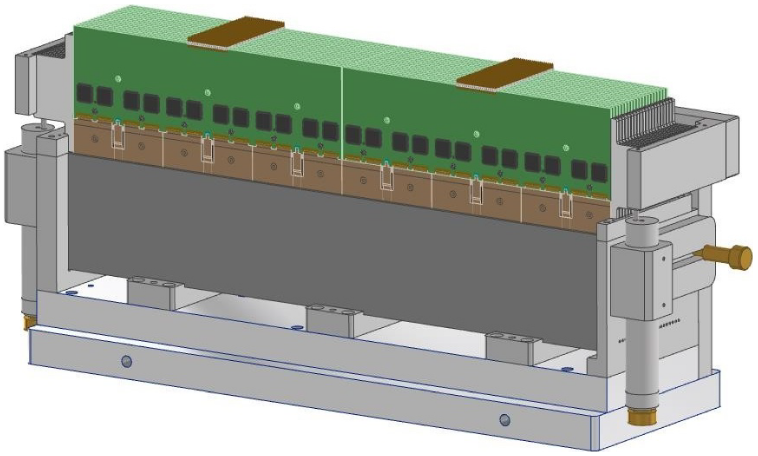


Figure 1: A drawing of the LUXE ECAL-P calorimeter. Frontend electronics boards (green) can be seen in aluminum frames on top of the sensors. Source: [2, ch. 5].

A conventional solution for circuit board identification involves placing some serial number as human-readable text on the board. This however has some drawbacks, such as a risk of the marking wearing off, or low visibility in a layered arrangement or low-light conditions. A more robust method is to store the board identifier electrically, in some non-volatile memory, and enable external readout of the value via a conveniently placed connector. This provides higher flexibility and durability, at the cost of some added complexity. As the identifying element, an inexpensive 1-Wire slave device can be

used, since the protocol requires each device to have a 64-bit, globally unique, factory-programmed and immutable address [3, sec. 6.1]. This address can serve as an identifier for the host board. The 1-Wire protocol allows communication via a single data line and includes built-in mechanisms for device discovery, which makes it highly suitable for this application.

The aim of this thesis was to design and build a prototype of a portable, hand-held 1-Wire identification code reader for performing circuit board identification. The device is tailored to the specific workflow of engineers working on the ECAL-P calorimeter. The thesis consists of three chapters.

- Chapter 1 contains a requirements specification for the device, as well as an overview of the technologies which can be used to fulfil the requirements.
- Chapter 2 details the developed device, outlining the system components and technical choices, and demonstrates its use.
- Chapter 3 shows details of the implementation, both electrical and software, while also describing the tests performed to ensure correct operation.

Chapter 1

Requirements and Technologies

The device designed in this thesis shall:

- be portable, its dimensions allowing it to be hand-held;
- make basic functionality available without the use of external equipment;
- allow to preview the read 1-Wire addresses on the go, as well as retaining or transmitting them for further reference;
- be usable and configurable as-is, without requiring reprogramming or dismantling for any supported maintenance operation.

These requirements led to the formulation of the specification described in this chapter, and selection of technologies to be used in the project.

1.1 Specification

Power and hardware

- The system shall be battery powered and rechargeable via USB-C.
- The firmware implementation should strive to optimize battery life.
- A physical power switch shall be present, to disconnect the battery when not in use.
- The primary user interface shall include a single push-button for control, and a display for showing device status. The display shall:
  - be sufficiently big to allow discerning characters at arm's length;
  - be readable from many angles and in the dark;
  - be able to display the 64-bit 1-Wire address in its entirety, as a 16-digit hexadecimal number, along with device status.

1-Wire

- The device shall be capable of acting as a 1-Wire master and performing a search procedure.
- A 3-pin connector for the 1-Wire bus shall be present, with 3.3 V power, data and ground pins.

- After a single click of the button, a 1-Wire search procedure for a single device shall be performed.
- The address of the found device shall be:
  - rendered on the local display;
  - logged via serial;
  - if no wireless connection is active, cached;
  - if a wireless connection is active, transmitted with all cached addresses.
- If no 1-Wire device is found, user should be informed of the condition.

Communication

- Logs containing both diagnostic data and 1-Wire addresses shall be transmitted via a USB-C serial interface.
- The device shall support IEEE 802.11 b/g/n wireless networking standards.
- DHCP (Dynamic Host Configuration Protocol) support shall be implemented, for automatic assignment of IP addresses.
- Upon initialization, the device shall attempt to associate with the most recently used Wi-Fi network, credentials stored in non-volatile storage.
- A panel consisting of a real-time data display and Wi-Fi credentials configuration form shall be served over HTTP, allowing access via a standard web browser.
- Real-time data shall be transmitted over WebSocket, to the most recently connected client.
- Long-pressing the button shall activate access point mode, creating its own Wi-Fi network for ad-hoc operation or configuration.
- Double-clicking the button shall re-initialize station mode, reconnecting to the configured network.

1.2 1-Wire

The 1-Wire interface [3, sec. 6.1] is a serial bus communication protocol designed by Dallas Semiconductor (now Analog Devices). Its main advantage is the fact that it uses only a single data line for communication on the whole bus. Hence, regardless of the number of devices connected to the bus, only two wires are needed – the data line (which also provides parasitic power to the slave devices via a pull-up resistor), and ground reference. The bus is controlled by a 1-Wire master, and supports communicating with one or more 1-Wire slave devices. The master searches and selects the slave device, initiating and synchronising the communication. For addressing purposes, every 1-Wire device has its own 64-bit address (ROM code), which is globally unique and cannot be changed.

1-Wire devices are often packaged in conventional transistor or integrated circuit packages. One example of such device is the DS18B20 thermometer [4], available in 8 pin surface mount packages (SOIC-8, uSOP-8) or a 3-pin TO-92 package. Another common example of a 1-Wire slave device is the iButton [5], which is in the form of a portable and

durable stainless steel pill with two contacts. When connected to a 1-Wire master, it gets powered over the data bus and can transfer its address to the master. This can be used for instance as a base for an electronic access control system, in which the iButtons function as keys for users, and the 1-Wire master verifies access rights.

1.2.1 Protocol

Data transmission in 1-Wire [3, sec. 6.2] is synchronous, since the transfer of each bit is initiated by a falling edge created by the master on the data line. Data is sent least significant (LSB) bit first. A transmission is started by the master with a reset pulse, setting the bus low for at least 480 µs. Each slave device on the bus responds after no more than 65 µs with a presence pulse, setting the bus low for 15...60 µs. Sending a bit from master to slave is initiated by the master with a short zero pulse on the bus (up to 15 µs). Afterwards, master holds the bus at the desired level, and slave samples the data. Sending a bit from slave to master is also initiated by the master with a short zero pulse, followed by a master sampling window in which the slave sets the desired bus level. The slave can only send data if it has been instructed to do so by the master via an appropriate command. Such communication protocol specification enables 1-Wire to reach a baud rate of up to 16.3 kBd (kilo baud). There is also an overdrive mode available, with 115 kBd speed [3, sec. 6.1].

Due to the simplicity of the protocol and a lack of additional control signals, 1-Wire devices need to be able to send and interpret commands. Such commands can be related to the addressing logic (ROM commands), or to a slave device's functionality. Table 1.1 shows selected commands supported by the DS18B20 temperature sensor, including generic ROM commands and some that are DS18B20-specific.

Table 1.1: Selected generic ROM and device-specific 1-Wire commands supported by the DS18B20 temperature sensor [4]. Each command is one byte long.

ROM commands	
Match ROM (`55h`)	followed by the slave's 64b ROM code; next command is addressed to the specified slave device
Skip ROM (`CCh`)	next command is addressed to all devices on the bus
Read ROM (`33h`)	read the slave's 64b ROM code (only when there's one slave on the bus)
Search ROM (`F0h`)	search procedure finds the 64b ROM codes of all slave devices on the bus
DS18B20 commands	
Convert T (`44h`)	initiate a single temperature conversion
Read scratchpad (`BEh`)	read all 9 bytes of the scratchpad – can be terminated by master with the reset signal
Write scratchpad (`4Eh`)	write 3 bytes of data into the scratchpad

1.2.2 Search procedure

A 1-Wire master can detect all slave devices connected to the bus via the search ROM procedure [3, sec. 6.4.2], analogous to a binary tree search where each bit of the address represents a branch. Figure 1.1 shows the structure of such a tree containing example 3-bit addresses.

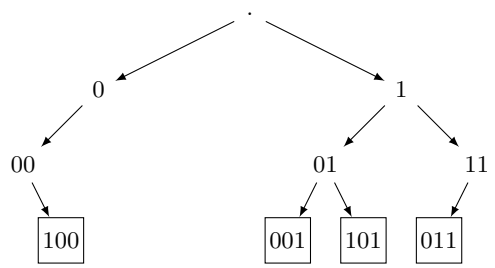


Figure 1.1: 1-Wire address tree for four 3-bit addresses: `100`, `001`, `101`, `011`. The search procedure traverses such a tree, visiting all leaves from left to right. Source: own work.

The procedure starts with a reset pulse. If any slave device responds, the master sends the Search ROM command (`F0h`). Afterwards, the master initiates a transfer of two bits – all slaves send first the least significant bit (LSB) of their address, and then the LSB negated. What reaches the master is a wired AND of the transmitted bits. Table 1.2 shows the interpretation of the received bits.

Table 1.2: Interpretation of a bit pair received by the master in a 1-Wire search cycle. Based on [3, tab. 6.2].

b	$\neg b$	**Interpretation**
0	0	conflict – both values have been transmitted
0	1	bit value is 0
1	0	bit value is 1
1	1	no slave device responded

A conflict signals that the node in the address tree at given bit position is a branching node. The master must remember such bit position and select the current path to take – for instance, always first assume 0 was read. The master then transmits the result of its interpretation onto the bus. Slave devices that detect a discrepancy between the bit they transmitted and the bit the master returned go into inactive mode, and remain so until a reset pulse is issued.

After repeating such a procedure another 63 times for consecutive bits of the ROM code, the master has read out the full 64 bit ROM code of one slave device. The master then issues a reset pulse and starts the procedure again, this time exploring another path (assuming 1 was read in the most recent unhandled branching node). This repeats until there are no more branching nodes to handle.

1.2.3 Suitability for the project

The 1-Wire interface is ideal for use in the project. Its 64 bit ROM codes are guaranteed to be globally unique, which enables reliable use as a module identifier. The fact that the ROM code handling and discoverability are built into the protocol enables easy implementation of identifier readout, done conveniently with two pins even if multiple chips were present on the bus. The only potential drawback of the protocol is its relatively low speed – at 16.3 kBd transmission of a byte takes about 0.5 ms, and the search procedure for a single device is around 100 ms. This however is not a problem, as the device will be used mostly on buses with one slave device, and will be operated manually, therefore a delay of up to a few hundred milliseconds is acceptable without being frustrating to the user.

1.3 Microcontroller

Since the device must respond to button events, handle incoming and outgoing communication via multiple interfaces and process network protocols, it requires some sort of processing unit to initiate and supervise this control logic. This can most easily be achieved by using a microcontroller unit (MCU).

An MCU is an integrated circuit containing one or more processor cores (CPU – central processing unit), memory and a set of peripherals for input and output (IO). The peripherals are dedicated hardware components for performing common tasks such as digital IO, serial communication, pulse width modulation, or timers. This eliminates the need for the processor core to perform demanding IO operations by itself – it can delegate this task to the peripheral, and wait for its completion (signaled by a bit in memory or an interrupt).

Microcontrollers can be very simple, such as the popular ATMega328P containing an 8-bit AVR core and a few peripherals [6], for implementing basic control logic and lightweight computations. They can also be much more advanced, such as the ESP32-S3 – with two 32-bit Xtensa LX7 cores, built-in 2.4 GHz Wi-Fi and Bluetooth Low Energy support, and a very rich peripheral system [7]. Figure 1.2 contains a functional block diagram of this microcontroller, demonstrating the variety of peripherals and subsystems it contains.

1.3.1 Interfaces and peripherals

The project requirements specify numerous interfaces to be used. Most of them are supported via dedicated peripherals, with some requiring additional implementation.

GPIO

The most basic interaction of a microcontroller with the outside world is via general purpose input/output (GPIO) [8]. When a physical pin is not used as a peripheral output, it can be configured to act either as a digital output, with the ability to set its voltage level to logical 0 or 1, or as a high impedance digital input, enabling the sampling of incoming logical signals. GPIO often offers features such as built-in pull-up or pull-down resistors, as well as the ability to set interrupts on IO events.

GPIO can be used for instance to control an LED (light emitting diode), or for reading a button state, as shown on figure 1.3. The push-button is in an active-low arrangement, with the input being pulled up by `R1` when the button is not pressed, and shorted to

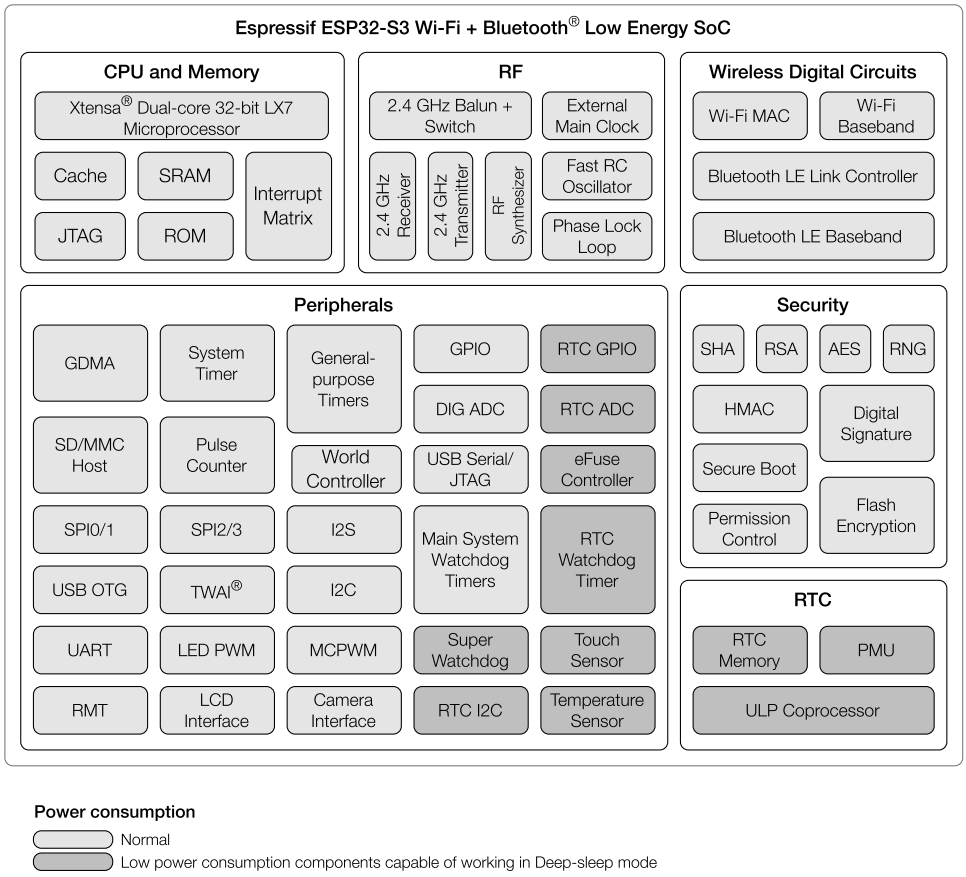


Figure 1.2: Functional block diagram of the ESP32-S3 microcontroller. Source: [7, p. 2].

ground in the opposite case. The LED is in an active-high arrangement, with `R2` serving as a current limiting resistor to prevent pin and LED burnout.

UART

UART [9], or universal asynchronous receiver-transmitter, is perhaps the most common serial data transfer protocol. It provides full-duplex communication at a moderate speed (commonly 9600 Bd or 115 200 Bd) via three lines: TX (transmitted data), RX (received data), and a ground reference. The communication is asynchronous, with no common clock signal, relying on start/stop bits and knowledge of the communication parameters (baud rate and frame structure). Data is transmitted in frames, which are structured as follows.

- Start bit – an idle UART data line is held at logical 1. The start bit is a logical 0 prepended to the transmitted data, to synchronize the communication.
- Data bits – after the start bit, the actual data is transmitted at the specified baud rate. 5 to 9 bits of data can be transmitted in a single frame.
- Parity bit – optional, used for validation of the transmitted data.
- Stop bits – UART data line is held at logical 1 for 1 or 2 bits before another frame can be transmitted.

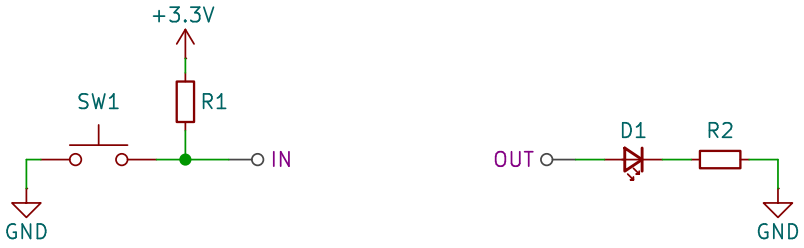


Figure 1.3: Example electrical connections for input/output with a GPIO – push button (left) and LED (right). Source: own work.

With the length of data segment set to 8 bits UART is perfect for transmitting character (byte) data, due to its simplicity and popularity. It is therefore often used as a log output from a microcontroller. UART to USB bridges are available in order to enable communication between the device and a computer with a standard USB connector.

I^2C

I^2C (Inter-Integrated Circuit bus) [3, sec. 2.1] is a synchronous serial bus protocol developed by Phillips. It uses two bidirectional lines – SDA for sending data, and SCL for transmitting the clock signal. Both lines should have pull-up resistors connected, to stay high when idle and allow the wired AND of incoming signals to be performed. An I^2C device can work either as a master, controlling the bus by transmitting the clock and start/stop sequences, or a slave. Each I^2C device model has its own 7 bit long address, to allow the master to direct its communication to the appropriate device. Usually it's possible to change part of the address to allow using mulitple devices of the same model on one bus. Typically, a microcontroller serves as the sole master on the bus, with multiple slave devices connected. This allows connecting multiple slave devices, using only two pins of the microcontroller.

1-Wire via RMT

The 1-Wire protocol is rarely supported in microcontroller peripheral systems. It therefore needs to be implemented on the software level. A common implementation is achieved by manually toggling and reading a digital IO pin at the appropriate times (bit-banging). Since the timing requirements of the 1-Wire protocol are in the order of tens of microseconds (100 kHz), this is easily achieveable with modern microcontroller clock speeds. It is however rather demanding for the CPU, and the timing might be skewed if the procedure is interrupted by a higher priority task.

Some microcontrollers offer versatile programmable peripherals that can take control of performing predefined IO sequences. This frees the CPU to perform other computations and leads to more deterministic IO. One example of such peripheral is the programmable input/output block (PIO) in the RP2040 microcontroller [10, ch. 3]. PIO consists of independent state machines, programmable with their own assembly language (pioasm). The instruction set is specialised for IO, focusing on determinism and precise timing. Such state machine can be programmed to execute 1-Wire data transfers without direct manipulation of the IO line by the CPU.

The ESP32-S3 offers a remote control peripheral (RMT), designed to send and receive

infrared signals for remote control [11, ch. 37]. It converts pulse codes stored in its memory into output signals, and reads input signals storing them as pulse codes in memory. It also supports modulating and demodulating the transmission with a carrier wave. This structure makes it very versatile and useful outside of infrared remote control – appropriately defined pulse codes enable the peripheral to perform 1-Wire communication. The RMT peripheral is used as the 1-Wire backend in the `onewire_bus` ESP-IDF component [12].

1.3.2 Networking

Wireless networking is a complex, multilayered process [13, sec. 1.4.1]. At the lowest level, the physical layer (PHY) needs to handle 2.4 GHz radio frequency (RF) signals, modulation and demodulation in real time. Next, the data link layer handles flow control, access to the communication medium and collisions (MAC – media access control). Further layers need to implement complex protocols for routing, packet loss and retransmission handling, such as the TCP/IP stack. On top of that, application layer protocols such as HTTP are required for usability in modern Web context.

PHY and MAC layers

In order to be able to fulfil the timing requirements of the PHY and MAC networking layers in a microcontroller, some hardware support is required. This can be accomplished in two main ways:

- Use of an external 2.4 GHz wireless interface – this is used for instance in the Raspberry Pi Pico W [14], a development board based on the aforementioned RP2040 microcontroller, which communicates via SPI (Serial Peripheral Interface) with the Infineon CYW43439 chip which handles the PHY and MAC layers.
- Integrated wireless submodule – for instance, the ESP32-S3 includes a built-in Wi-Fi radio and baseband, as well as a Wi-Fi MAC controller, implementing the full 802.11b/g/n protocol.

Both those solutions however still require external circuitry for transmitting the generated radio frequencies. This is difficult, since such high frequencies are very sensitive to printed circuit board (PCB) layout, and need to be certified for electromagnetic compatibility before use. To ease this process, Espressif (manufacturer of the ESP32-S3) sells complete pre-certified modules. An example of such a module is the ESP32-S3-WROOM-1 [15], containing an on-board antenna, RF matching circuitry and a shielding case for reducing interference. This enables rapid prototyping and reduces costs associated with developing a Wi-Fi-enabled product.

TCP/IP

In order to enable connecting of independent networks in a manner that's seamless, the TCP/IP stack has been developed [13, sec. 1.4.2]. The Internet Protocol (IP) is responsible for handling the network layer, managing addressing and routing to ensure successful packet delivery between networks. Building upon this, the Transmission Control Protocol (TCP) in the transport layer allows two entities on the network to communicate via a reliable data stream.

Due to the protocols' complexity, it is rather cumbersome to implement them manually. Moreover, due to the resource constraints of microcontrollers, special optimizations are needed for memory and speed efficiency. A common choice for an existing TCP/IP stack

implementation is the lwIP [16] library, which focuses on optimizing memory usage but still implementing all features of TCP.

Application layer protocols

HTTP (Hypertext Transfer Protocol) [17] is an application layer protocol sent over TCP, used for transmitting resources such as HTML in all Web applications. It follows a client-server model, with a client making requests to a server and waiting for the response. The protocol is stateless. HTTP messages (in protocol version 1.1 and below) are sent in text form and designed to be human readable. Examples of such messages are shown in figure 1.4. A HTTP request contains the following elements:

- the request method, for instance:
  - `GET` – requests a resource, should contain no data;
  - `POST` – sends data to a resource;
  - `PUT` – replaces a resource with the contents of the request;
  - `DELETE` – deletes a resource.
- the path of the target resource;
- the HTTP version;
- headers with additional information (optional);
- for some methods (e.g. `POST`, `PUT`), a body with sent data.

A response to the request contains:

- the HTTP version;
- a status code;
- a message associated with the status code;
- headers (like in request);
- for most methods (e.g. `GET`, `POST`, `PUT`, `DELETE`), a body with sent data.

Since HTTP is request and response based, there is no direct way for the server to transmit data without the client requesting it, which is sometimes required for dynamic content updates. WebSocket has been created to bridge this gap. It is a protocol for initiating a TCP connection between client and server in a Web environment [18, ch. 1]. A WebSocket is initiated by the client by sending a `GET` request with the appropriate headers to the server. The server then validates the request, initializes the socket, and completes the handshake by sending a response with the *101 Switching Protocols* status code. Afterwards, either the client or the server can choose to transmit data via the socket at any time.

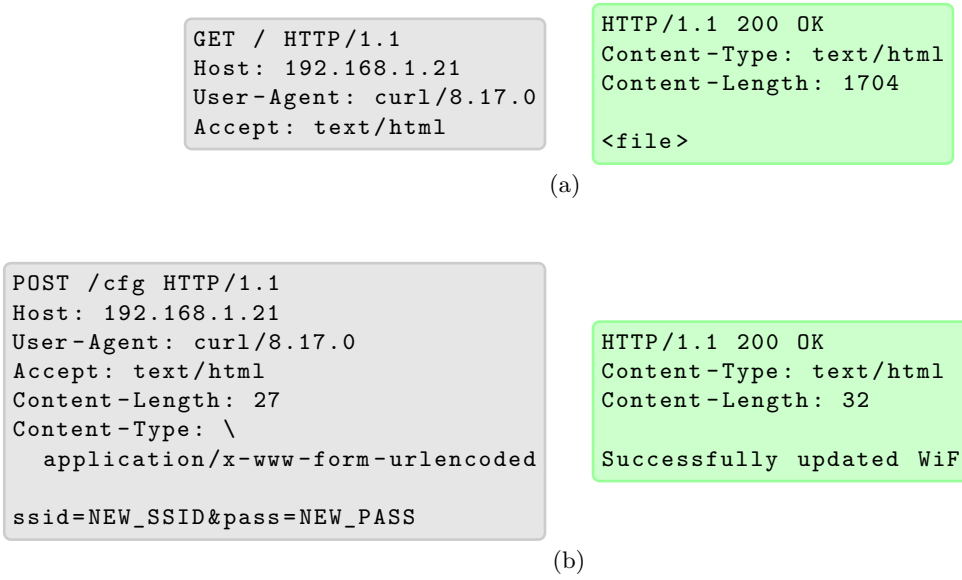


Figure 1.4: Example HTTP request (left) and response (right) pairs. Request (a) is a `GET` request to the `/` path (`GET /`). Its response contains HTML data (denoted here with `<file>`). Request (b) is a `POST` request to the `/cfg` path (`POST /cfg`), containing a body with some configuration in `key=value&...` form. Its response contains text informing of successful configuration. All messages contain the appropriate headers, informing i.a. of body content type and length. Source: own work.

1.3.3 Programming

Programming a microcontroller at the lowest level involves directly manipulating bits in dedicated registers of memory-mapped peripherals. This is rather cumbersome, and requires a deep understanding of the peripheral's inner structure. To ease this process, vendors often provide hardware abstraction layers that offer functions with readable names for performing low-level tasks. Even with this hardware abstraction however, multitasking is still difficult and requires careful management of timers and interrupts.

For embedded systems that need to perform many concurrent operations, it may be beneficial to use a basic operating system. A common choice is Amazon FreeRTOS [19, ch. 1], comprised of a real time kernel and a set of libraries to complement it. A unit of execution in FreeRTOS is called a task, and there exist flexible mechanisms to assign priorities to tasks. Moreover, FreeRTOS provides primitives for communication and synchronization between tasks, such as queues, semaphores and mutexes. It is therefore a complete solution to facilitate development of efficient, concurrent embedded applications, mixing hard and soft real time requirements.

For programming ESP32-series microcontrollers, the ESP-IDF (Espressif IoT Development Framework) is available. It is based on FreeRTOS and organized into components [20]. Built-in components include basic hardware abstraction, Wi-Fi drivers, the TCP/IP stack implementation or a HTTP server. Additional components are available online via the ESP Components Registry and can be integrated into the project using the `idf.py` tool. Configuration of all the used components is available via the `idf.py menuconfig` terminal user interface. The ESP-IDF is a very powerful solution, offering all the components required to build firmware for a Wi-Fi-enabled device.

1.4 Power

Modern microcontrollers are usually powered by 3.3 or 5 volts DC (direct current). The ESP32-S3-WROOM-1 module runs on 3.3 V accepting voltages within the ±0.3 V range of the nominal voltage [15, sec. 6.2]. In order for the system to achieve full portability, the microcontroller needs to be powered by a battery.

According to an extensive survey of battery options for powering IoT (Internet of things) devices [21], the choice of appropriate battery type is heavily dependent on the protocol requirements. For a device supporting Wi-Fi communication, a battery with a high energy density is required due to high power consumption during data transmission. While popular alkaline batteries are often favored due to their safety and low cost, they are unsuitable for such high demand applications. Moreover, since they are not rechargeable, they require frequent replacement and are therefore a less sustainable solution. Lithium-Ion (Li-ion) and Lithium-Polymer (LiPo) batteries emerge as a better alternative, offering higher energy density an rechargeability. Those technologies however pose some safety risks, with their flammability as the primary concern, requiring additional protection circuitry. LiPo batteries are contained in more flexible casing that Li-ion, making them more susceptible to mechanical damage, but also allowing them to be contained in a thinner form factor.

To enable a LiPo-powered device to be recharged, a dedicated control module is required to oversee the charging process. It must provide optimal charging voltage and current to prevent degradation, and finalize the process when the battery capacity is reached. A popular solution to this problem are the widely-available LiPo charger modules based on the TP4056 integrated circuit [22]. Such modules accept 5 V input via USB, and provide the appropriate 4.2 V charging voltage, limiting the current to 1 A. Upon completion of the charging cycle, an on-board LED is illuminated.

The nominal output voltage of a LiPo battery is 3.7 V, however the actual voltage ranges from 3.0 V (fully discharged) to 4.2 V (fully charged) [23]. This is outside of the acceptable range (3.0 to 3.6 V [15, sec. 6.2]) for the mentioned ESP32 module, and therefore requires regulation. Due to the dropout voltage of step-down regulators, to be able to use the battery in its full voltage range a buck-boost converter is required, which can maintain a steady output voltage for inputs both lower and higher than the desired voltage. Ready-made modules, such as the Pololu S8V9F3 DC-to-DC converter module [24], are available. The module keeps a constant 3.3 V output voltage for input voltages from 1.4 to 16 V, however its minimum startup voltage is 2.7 V. This is still enough for it to cover the full voltage range of the LiPo battery. Its maximum output current capability for input voltages in the range of 2.7 to 4.2 V is practically equal to 1.5A, which is sufficient even for the ESP32-S3-WROOM-1's Wi-Fi spikes (up to 355mA [15, sec. 6.4]).

Chapter 2

The OWL Device and Its Usage

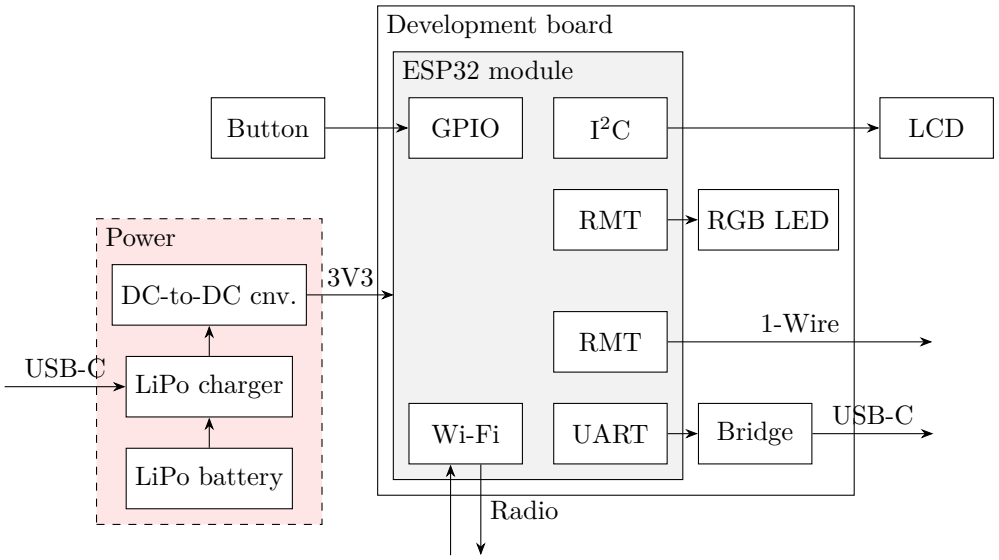


Figure 2.1: Block diagram of OWL. Source: own work.

The device developed in this thesis has been named OWL, which stands for **O**ne **W**ire **L**ocator. Figure 2.1 shows a block diagram of the device. OWL's user interface includes a push button for device control, as well as an LCD (liquid crystal display) and RGB LED (red-green-blue light emitting diode) for direct communication of device status. For external connectivity, OWL features a USB-C connector for serial log transmission, a 1-Wire header for performing bus readout and Wi-Fi capabilities for wireless control. The system is based on an ESP32-S3 development board, with Wi-Fi capabilities and a rich peripheral system. OWL is battery powered, with appropriate circuitry for charging the battery via a dedicated USB-C connector, as well as a DC-to-DC converter to convert the voltage to that required by the development board.

2.1 System components

As the main control unit for OWL, the ESP32-S3-WROOM-1-N8R8 module has been selected [15]. It meets all the requirements of the project, with a powerful dual-core CPU, 512 kB SRAM and 8 MB flash, a rich peripheral system and support for the Wi-Fi

802.11b/g/n standards. It contains an on-module Wi-Fi antenna, and development boards with the module are available, making it highly suitable for rapid development and prototyping. The specific development board used for OWL (Waveshare ESP32-S3-DEV-KIT-N8R8 [25]) contains, apart from the ESP32 module, an RGB LED and a UART-to-USB bridge, used for displaying basic status updates and transmitting logs via USB-C respectively.

The board is powered via the 3.3 V pins, providing direct power to the ESP32 module. A LiPo battery has been selected, due to its high energy density (required for a device with Wi-Fi support) and small size (keeping in mind the portability requirements), with a capacity of 1050 mAh. The voltage from the battery passes through a TP4056 based LiPo charger module, enabling charging of the device and containing protection against excessive discharging. It is then converted to the required 3.3 V by use of the Pololu S8V9F3 DC-to-DC converter module [24]. This arrangement is sufficient for powering the ESP32 development board, as mentioned in the previous chapter.

Taking into account the readability requirements, a 16x2 characters liquid crystal display (LCD) has been selected. It was bigger than most available OLED (organic light-emitting diode) displays, while still remaining compact enough to fit in a hand-held device. The specific display used in OWL is the Gravity I2C 16x2 Arduino LCD with RGB Font Display (SKU DFR0554) [26]. It's differentiating feature is the RGB backlight that is used for communicating the device status even if the display text isn't directly visible. More importantly, it supports 3.3 V operation, which means it is aligned with OWL's power scheme. It is controlled via the I^2C interface, with the display and backlight controllers being separate devices on the I^2C bus. This enables easy integration with the ESP32 module via the I^2C peripheral, using only 2 pins for control of both devices.

The OWL firmware has been implemented in C, using the ESP-IDF. The IDF has been selected due to it being more flexible and professional than the Arduino ecosystem, while still providing drivers for all required hardware subsystems, implementations of network protocols and a registry of components for common tasks. This facilitated the creation of modular and configurable firmware, ensuring reliability and maintainability. The firmware can be configured before compilation by use of a unified configuration menu provided by ESP-IDF, as a separate OWL component.

2.2 Usage

2.2.1 Basic usage

Figure 2.2 shows OWL fully assembled and powered on. Before powering on OWL, it is necessary to ensure all the modules are connected. The battery must be connected to the 2-pin header ①, with the orientation corresponding to that indicated on the adjacent charger module ②. The LCD must be connected to the 4-pin header ③, with power being the outermost wire. OWL can then be powered on using the slider switch ④. Immediately after power up, the on-board RGB LED ⑤ should flash yellow, and then green after successful initialization of all subsystems. The status page should appear on the LCD, informing of the device's mode and status.

OWL is controlled using the red push button ⑥ located at the center of the board. Basic usage of OWL consists of making contact with a 1-Wire chip via the angled 3-pin header ⑦ and pressing the button to read its address. The chip's 1-Wire address is then read and

- displayed (with information of transmission status);

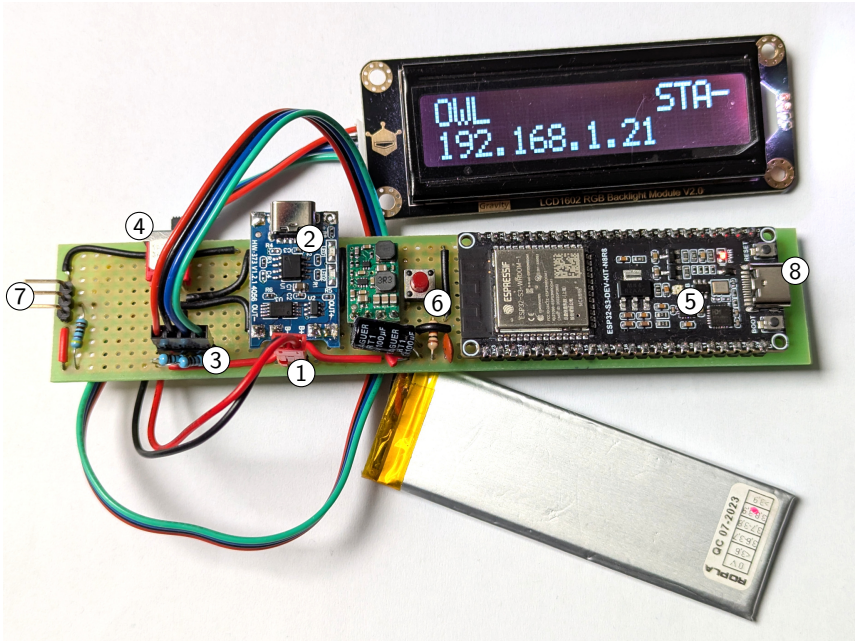


Figure 2.2: OWL fully assembled. Components for user interaction: 1 – battery connector; 2 – charger module and connector; 3 – LCD connector; 4 – power switch; 5 – RGB LED; 6 – main button; 7 – 1-Wire connector; 8 – log output. The display shows the status page for OWL in STA mode, connected to a Wi-Fi network, with the 192.168.1.21 IP address assigned and no WebSocket connection active. Source: own work.

- logged to the serial output;
- if no WebSocket connection is active, stored in memory;
- if a WebSocket connection is active, transmitted along with all previously stored addresses via WebSocket (to most recent connection only).

During 1-Wire activity, the LED ⑤ should flash cyan.

2.2.2 Serial logs

OWL uses UART to emit logs via a serial to USB bridge. They can be used for debugging purposes or to extract and store 1-Wire addresses. In order to be able preview the logs, it is necessary to connect the device to a computer via the USB-C connector present on the ESP32 development board ⑧. To use this mode, the battery power switch needs to be turned off, since OWL is then powered via the USB-C connector and the voltage regulator on the development board. There are two main options for accessing the serial data stream:

- using `idf.py monitor` – if ESP-IDF is installed, this tool (run in the source code project directory) will auto-detect the connected device, reset it and display the logs with color codes based on severity;
- using any serial monitor with the following serial communication parameters:
  Baud rate: 115200
  Data bits: 8

Parity: none
Stop bits: 1
Flow control: none
The logs will not be color coded, and a device reset will need to be performed manually by pressing the on-board `RESET` button.

Figure 2.3 shows some example logs taken from the device. Note that logs from OWL are mixed with logs from vendor drivers. The module from which a log was emitted is indicated by the tag, placed directly after the severity and timestamp information.

```
I (34827) esp_netif_handlers: sta ip: 10.101.182.24, mask: 255.255.255.0, gw: 10.101.182.125
I (34827) owl_display: To display (color 202020, -1 ms):
OWL         STA-
10.101.182.24
I (34827) owl_wifi: Got IP: 10.101.182.24
I (34827) owl_display: To display (color 00ff00, 3000 ms):
Connected
EXAMPLE_SSID
I (41857) wifi:<ba-add>idx:1 (ifx:0, 46:2a:6a:09:85:8e), tid:7, ssn:2, winSize:64
I (51587) owl_display: To display (color 202020, -1 ms):
OWL         STA+
10.101.182.24
I (51587) owl_http_server: WS handshake done (fd = 59)
I (61497) owl: Found device #0: 879D418E34646128
I (61497) owl_display: To display (color 00ffff, 5000 ms):
879D418E34646128
SENT 1
:
```

Figure 2.3: Example logs taken from the device. Sequence of captured operations: (1) connect to `EXAMPLE_SSID` network; (2) access OWL panel in browser (3) read 1-Wire address. Source: own work.

2.2.3 Wi-Fi interface

The Wi-Fi interface is an extension of the basic functionality of the device. OWL offers two modes of Wi-Fi operation:

- STA mode - OWL serves as a Wi-Fi station, and can associate with an existing Wi-Fi network;
- SoftAP mode - OWL serves as a Wi-Fi access point, creating its own Wi-Fi network.

After powering up the device, it will start out in STA mode and attempt to associate with the most recently used Wi-Fi network (yellow backlight on LCD). If it succeeds, a success message with the network's service set identifier (SSID, green backlight) will be displayed on the LCD, followed by the status page (white backlight). If it fails, the status page will be displayed (red backlight). In order to re-attempt association to the network, user needs to double-click the button (LED ⑤ should flash white). In order to reconfigure the Wi-Fi credentials, SoftAP mode can be enabled by long-pressing the button (LED ⑤ should flash yellow).

In order to leverage the wireless capabilities of OWL, it is necessary to connect to its HTTP server via a web browser. This is done in different ways depending on the Wi-Fi mode:

- STA mode – on a device connected to the same Wi-Fi network as OWL, enter the IP address displayed on the status page into a web browser's address bar;

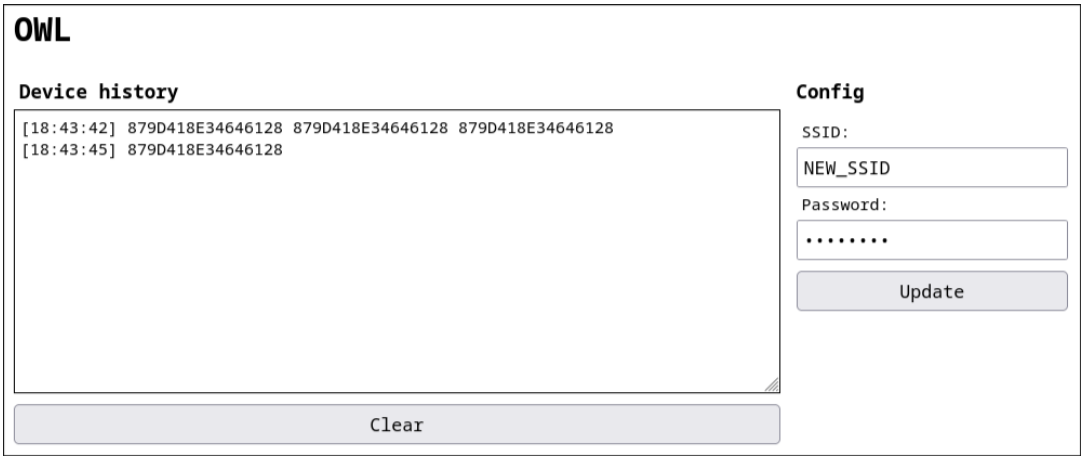


Figure 2.4: Example use of the OWL panel. The `Device history` panel contains two entries – one sent after reading a 1-Wire address after two addresses had been cached, the other sent after reading an address with no cached addresses (all read from one DS18B20 thermometer). The `Config` form contains example data for reconfiguring the connection. Source: own work.

- SoftAP mode – connect a device to the Wi-Fi network created by OWL (credentials displayed on the status page) and enter `192.168.4.1` into a web browser's address bar.

Upon successful connection to the device, the OWL panel (shown in figure 2.4) is served to the client and a WebSocket connection is initiated (replacing the previous WebSocket connection), regardless of the Wi-Fi mode. Afterwards, every read 1-Wire address appears in the `Device history` text box (left hand side), along with a timestamp indicating when the information arrived. The box can be resized or cleared if needed. Another important feature of the OWL panel is the ability to reconfigure the Wi-Fi credentials used in STA mode. To do this, the new credentials need to be entered into the `Config` form (right hand side) and confirmed by pressing the `Update` button.

2.2.4 Status page

First line of the status page always starts with the name OWL. The remaining contents depend on the mode and status of the device, as shown in figure 2.5.

- In STA mode with a successful Wi-Fi connection, first line is terminated with the code `STA`, followed by an indicator of whether a WebSocket connection is active (`+`) or not (`-`), represented on figure 2.4 as a question mark. The second line contains the IP address assigned to the device. The backlight is white.
- In STA mode with no Wi-Fi connection, first line is terminated with the code `NC` (not connected). The second lines contains the SSID of the network to which association was attempted. The backlight is red.
- In SoftAP mode, first line is terminated with the code `AP`, followed by the WebSocket indicator (as in STA mode). The second line contains the password of the created Wi-Fi network. The backlight is yellow.

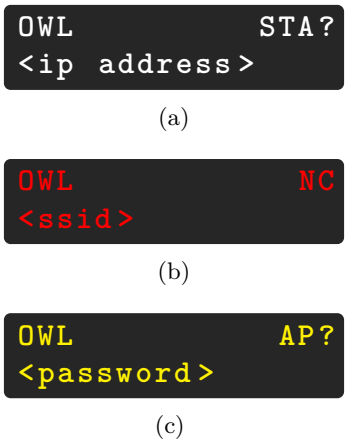


Figure 2.5: Layout and color of the status page in modes: STA with Wi-Fi (a), STA with no connection (b), SoftAP (c). The question mark symbolizes the WebSocket connection indicator (`+` or `-`). `<...>` placeholders are replaced with the appropriate data. Source: own work.

2.2.5 Firmware reconfiguration

In rare cases of hardware changes, or issues with SoftAP mode connectivity, the firmware can be reconfigured and recompiled. This shouldn't be necessary for normal operation. The ESP-IDF provides a unified way of configuring all components of a project via `idf.py menuconfig`. Upon entering this command in the firmware project directory, a terminal user interface appears with a multitude of options for ESP32 subsystems and ESP-IDF components. OWL options can be found by navigating to `Component config --> OWL`. The available options are:

- `Development mode` – allows to hard code Wi-Fi credentials in `secrets.inc` file, to ease testing while re-flashing the board often;
- `WiFi SSID` – SSID of the network to connect to in STA mode;
- `WiFi password` – password of the network to connect to in STA mode;
- `SoftAP SSID` – SSID of the network created in SoftAP mode;
- `SoftAP password` – password of the network created in SoftAP mode;
- `SoftAP WiFi channel` – Wi-Fi channel to use in SoftAP mode;
- `SoftAP max connection` – maximum number of devices that can connect to the created network in SoftAP mode;
- `LED GPIO` – number of the pin used for controlling the LED;
- `Button GPIO` – number of the pin used for controlling the button;
- `OneWire bus GPIO` – number of the pin used for controlling the 1-Wire bus;
- `Use LCD` – use the LCD;
- `LCD display I2C SDA pin` – number of the pin used as the I^2C SDA line for controlling the LCD;

- `LCD display I2C SCL pin` – number of the pin used as the I^2C SCL line for controlling the LCD.

Figure 2.6 shows an example configuration of the device, with pin numbers corresponding to those on the assembled device.

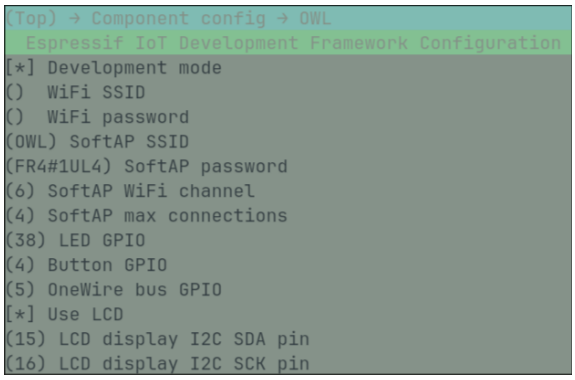


Figure 2.6: Example configuration of OWL. Source: own work.

Chapter 3

Implementation Details and Tests

3.1 Electrical connections

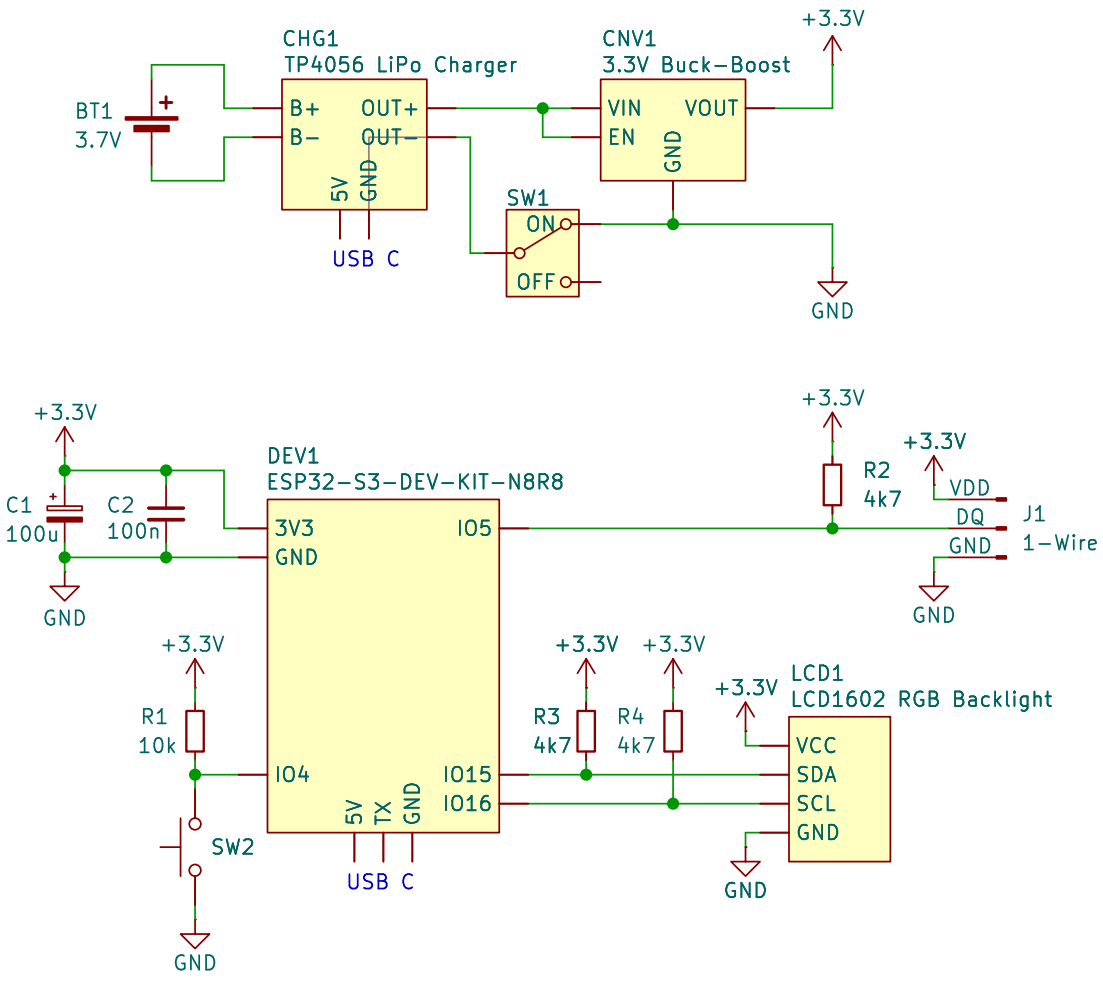
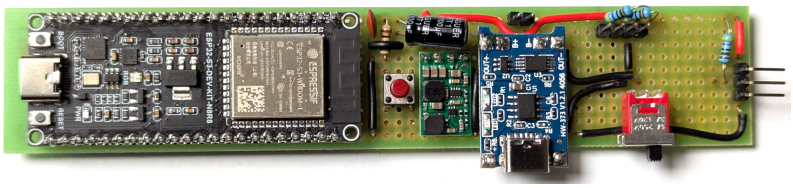


Figure 3.1: Electrical schematic of the device, with modules represented as blocks. Source: own work.

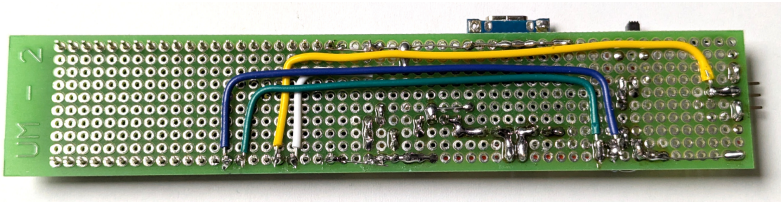
Figure 3.1 shows the electrical schematic of OWL. The top part of the schematic contains the power supply module, providing stable 3.3 V power to all subsystems from the

LiPo battery `BT1`. The used modules are connected according to their pin descriptions, with the enable pin of the buck-boost converter hardwired to positive. Battery charging is performed via the on-board USB C connector of the LiPo charger module `CHG1`. `SW1` serves as the power switch and is added to conserve battery power. The bottom part of the schematic contains the ESP32 development board along with all control logic subsystems. In light of issues with the microcontroller rebooting during Wi-Fi current spikes, decoupling capacitors `C1` and `C2` have been connected to the board's power pins, to be placed as close as possible. The ESP32-S3 has a GPIO matrix, allowing to route peripheral outputs to any pin [11, ch. 6]. The pins have therefore been selected solely with easy assembly in mind. The main push button `SW2` is connected to `IO4` microcontroller pin, pulled up by a 10 kΩ resistor `R1`. Pull up resistors `R2`, `R3`, `R4` (all valued 4.7 kΩ) are also connected to the 1-Wire (`IO5` microcontroller pin) and I^2C (`IO15`, `IO16` microcontroller pins) lines, as per protocol requirements.

The device is assembled using generally available through-hole components on a 28 × 147 mm perfboard (10 × 54 holes with 2.54 mm spacing). Figure 3.2 shows detailed top and bottom views of the assembled device (battery and display are disconnected for clarity), with clearly visible modules and connections. The dimensions of the board enable easy handling, achieving the requirement of hand-held portability. The button is positioned in a way that is convenient to access with the thumb when holding the device.



(a)



(b)

Figure 3.2: Top (a) and bottom (b) views of the assembled device. On the top view all modules are visible – ESP32 development board (black), voltage converter (green), LiPo charger (blue). The bottom view shows all the connections and solder bridges. Source: own work.

3.2 Source code

OWL's firmware is implemented in C using ESP-IDF v5.5.1 and is available on GitHub (`https://github.com/frun36/owl`). Portions of the codebase, especially the initialization boilerplate, use Espressif's standard ESP-IDF examples [27] as a foundation.

3.2.1 Firmware modules

The code is built from modules consisting of a `.h`/`.c` file pair, each module responsible for one aspect of OWL's operation. This clear separation of concerns makes the code easily maintainable and extensible. To further enforce clean code practices, all modules are prefixed with `owl_`, and all public functions with the entire component name. This simulates the concept of namespaces from higher-level languages. Below is a list of all the modules, with a description of their functionality.

`owl_button`

The `owl_button` module is a wrapper over ESP-IDF's `button` component. It provides:

- `owl_button_init` – a button initialization function;
- `owl_button_event_t` – definitions for button events that will be used by the device (single click, double click, long press);
- `owl_button_event_queue` – a FreeRTOS queue to transmit information about the events.

`owl_color`

The `owl_color` module consists of only a header file, with utilities for representing color:

- `owl_rgb_t` – a struct storing an RGB (red-green-blue) color value;
- `owl_color_t` – an enum with basic color definitions;
- `owl_rgb` – a function for representing those basic colors as RGB.

These are used in control functions for the RGB backlight of the LCD and the on-board RGB LED.

`owl_display`

The `owl_display` module is an abstract framework for connecting different displays to the device. It supports displaying two-line messages (16 characters in each line) with a color code. It provides:

- `owl_display_init` – a function to initialize the display;
- `owl_display` – a function to display a message for some time;
- `owl_display_update_status` – a function to request update of the status page.

Conditional compilation can be used to specify the concrete display that will be used. Currently only the aforementioned LCD with RGB backlight is supported, but this can easily be extended by implementing a driver for a different display compatible with the `owl_display` component's interface.

The display logic is based on a FreeRTOS task and queue, as shown in figure 3.3. By default, the status page (containing information about current device wireless mode, connection status and IP address) is shown. When user calls the `owl_display` function, a request to display the specified two-line message with a color code and duration is sent to the task via the queue. The display task, after receiving the request, displays the message

until either a new message arrives, or the specified duration has passed. Then it reverts back to the status page. When user calls the `owl_display_update_status` function, information about the status is gathered and a display request is sent with time set to -1 (special value for no timeout). The backlight for status is set to a much dimmer color, to conserve battery power.

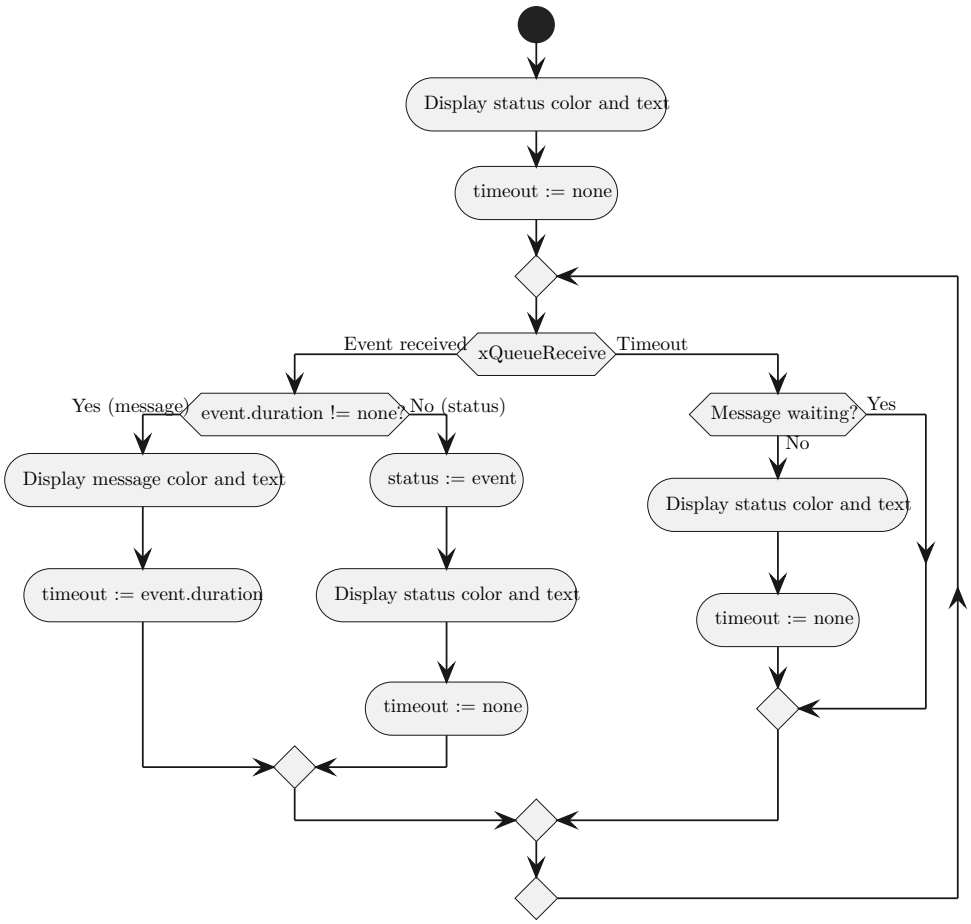


Figure 3.3: Display task logic. Initially the status page is displayed, and the queue waits for a new message with no timeout. If a new message arrives, it gets displayed and the queue waits for a new message with timeout set to duration of the message. If no new message arrives, the status page is displayed. Source: own work.

`owl_lcd`

The `owl_lcd` module is a driver for the LCD with RGB backlight. It provides:

- `owl_lcd_init` – a function to initialize the I^2C bus and communication with both the display and backlight modules;
- `owl_lcd_set_backlight` – a function to set the backlight color;

- `owl_lcd_write` – a function to write a string on the specified display line;
- `owl_lcd_clear` – a function to clear the display.

These functions are then called in the `owl_display` component to provide concrete execution of the abstract requests it provides.

There is no ESP-IDF component for controlling this particular display model. Also, documentation for the display is quite sparse and imprecise. Fortunately, an Arduino library was available on GitHub [28]. It was possible to extract the communication protocol, including driver I^2C addresses, available commands and their format, timing constraints, and port the required functionalities to ESP-IDF.

`owl_led`

The `owl_led` module is responsible for controlling the on-board RGB LED (WS2812B). It is a wrapper over Espressif's `led_strip` IDF component [29] configured for a single LED on the strip, and using the RMT peripheral as a backend. It provides:

- `owl_led_init` – a function to initialize the component and RMT peripheral;
- `owl_led_set` – a function to light up the LED with provided color;
- `owl_led_clear` – a function to turn the LED off.

The LED is used as the most low-level status display of the device and is flashed in different colors when events occur.

`owl_onewire`

The `owl_onewire` module is responsible for the 1-Wire address readout. It is a wrapper over Espressif's `onewire_bus` IDF component [12], using the RMT peripheral as the backend. It provides:

- `owl_onewire_init` – a function to initialize the component and RMT peripheral;
- `owl_onewire_search` – a function to perform a 1-Wire address search of up to `max_devices` devices and store their addresses in a buffer.

`owl_wifi`

The `owl_wifi` module is a relatively complex module, responsible for initializing and configuring the Wi-Fi connection, switching between station and SoftAP modes, as well as handling Wi-Fi events and providing info about the status. It provides:

- `owl_wifi_init` – a function to initialize the Wi-Fi subsystem and all other required subsystems;
- `owl_wifi_configure` – a function to configure both station and SoftAP interfaces;
- `owl_wifi_sta` – a function to move into station mode;
- `owl_wifi_ap` – a function to move into SoftAP mode;
- `owl_wifi_status_t` – an enum representing current status of Wi-Fi;
- `owl_wifi_get_status` – a function to get current status of Wi-Fi;

- `owl_wifi_get_ip_str` – a function to get a string representation of the IP address assigned to the device when in station mode.

First, the `owl_wifi_init` function needs to be called – it initializes the non-volatile storage (NVS) component to store the configured Wi-Fi credentials, initializes the Wi-Fi subsystem itself (for both STA and SoftAP modes), creates an event loop end registers event handlers for Wi-Fi and IP events. It also creates a mutex to protect shared access to variables storing the Wi-Fi status, which are used for display purposes. Then, `owl_wifi_configure` applies configuration for both STA and SoftAP modes, including the Wi-Fi credentials. Afterwards Wi-Fi can be started or restarted on demand, by calling either `owl_wifi_sta` or `owl_wifi_ap`, in STA or SoftAP mode respectively – each perform a complete restart, including resetting the WebSocket connection made in previously used mode. The `owl_wifi_get_status` and `owl_wifi_get_ip_str` functions are used to display the current Wi-Fi status, and provide safe concurrent access to the status variables.

`owl_http_server`

The `owl_http_server` module is responsible for handling the HTTP server running on the device, which serves the OWL panel, and for handling WebSocket connections used for sending read 1-Wire addresses and displaying them in the panel live. The HTTP server and WebSocket low-level logic is handled by built-in ESP-IDF components. It provides:

- `owl_http_server_init` – a function to initialize the HTTP server;
- `owl_ws_is_connected` – a function to check whether there is an active WebSocket connection;
- `owl_ws_reset_fd` – a function to forget the currently active WebSocket connection;
- `owl_ws_send` – a function to send data over WebSocket.

The module also handles all logic related to the `index.html` file – an independent application containing the OWL panel, sent to the client to be executed in a web browser. It is stored in the flash memory of the ESP32 module, using a basic filesystem provided by the ESP-IDF called SPIFFS. Since it is rather small (1.7 kB), it gets loaded in its entirety into memory on device power up, to be served by the HTTP server implementation. The `index.html` file defines the structure of the user interface, with some lightweight styling. It also contains a short JavaScript script to initiate a WebSocket connection on startup and display the received data.

The `owl_http_server_init` function initializes the server itself, registers endpoint handlers and loads the `index.html` file. The core logic is defined in endpoint handlers, namely:

- `GET /` – serves the `index.html` file;
- `POST /cfg` – used for uploading (as request body) and applying updated Wi-Fi configuration (SSID and password);
- `GET /ws` – a handler for WebSocket connections, responsible for opening and closing the connection, storing the most recent WebSocket file descriptor for use in other functions.

Data is sent over WebSocket in the `owl_ws_send` function (using ESP-IDF's built-in WebSocket capabilities) to the last stored file descriptor. A stored file descriptor value of -1 means no WebSocket connection is active.

`owl_main`

The `owl_main` module ties the whole application logic together. It initializes all previously mentioned modules, and creates a FreeRTOS task (`owl_task`) that handles button events and performs the desired actions.

3.2.2 Event handling

Apart from the main function, which initializes the modules, all other operations are performed in response to an event – caused either by the user, or the Wi-Fi connection. The most important events handled by the application are:

- button: click – read and handle 1-Wire address;
- button: 2x click – move Wi-Fi to STA mode, attempt connect 3x;
- button: long press – move Wi-Fi to SoftAP mode;
- Wi-Fi: disconnected – attemp reconnect 3x;
- Wi-Fi: got IP address – display connection data;
- HTTP: `GET /` – serve index.html;
- HTTP: `POST /cfg` – update Wi-Fi credentials;
- HTTP: `GET /ws` – initialize and save WebSocket connection.

Some other Wi-Fi events are also handled to produce debug logs, which are not described here for clarity. Button events are transferred via a FreeRTOS queue to the `owl_task` task, which then calls functions from all involved modules. Wi-Fi events are handled directly in callback functions in the `owl_wifi` module, which then perform other Wi-Fi operations and display logic.

Figures 3.4-3.6 show sequence diagrams of the handling logic for the main events. The participants of the diagram are separated into a few color-coded categories:

- red modules are ESP-IDF modules, imported as ESP-IDF components, potentially containing layers of abstraction with multiple FreeRTOS tasks;
- yellow modules are OWL task modules, with FreeRTOS tasks that have been created and are managed entirely in the OWL source code;
- green modules are OWL basic modules, containing abstraction layers over hardware operations, with no inner FreeRTOS tasks.

Many operations in the system are executed asynchronously, e.g. via FreeRTOS queues. This is always indicated on the diagrams by the lack of a return arrow. On the other hand, functions which operate synchronously always have this arrow.

Figure 3.4 shows the handling of all supported button events – single click, double click and long press. All button events are transmitted via a FreeRTOS queue to the main task.

- A single click initiates the 1-Wire search procedure, lighting up the LED (cyan) during the search. The result is then displayed on the display and sent via WebSocket or stored (depending on the connection status).

- A double click reinitializes/returns to STA mode, blinking the LED (white).
- A long press reinitializes/enables SoftAP mode, blinking the LED (yellow).

Figure 3.5 shows the handling of HTTP requests sent to any of the supported endpoints, namely `GET /`, `POST /cfg`, `GET /ws`.

- `GET /` – fetches the `index.html` file contents from memory and sends them to the client.
- `POST /cfg` – extracts the specified SSID and password from request body and updates the STA mode config.
- `GET /ws` – handles a WebSocket connection request using the ESP-IDF WebSocket implementation.

Figure 3.6 shows the handling of Wi-Fi events. The events are emitted by the IDF component itself after some operations are performed to enable appropriate handling. A Wi-Fi connection attempt is made in response to either the `STA_START` (initial connection, displays status update) or `STA_DISCONNECTED` (reconnection) events. The connection attempts are indicated on the display and counted. Their results are indicated by emission of events – `STA_CONNECTED` or `STA_DISCONNECTED`. If 3 unsuccessful attempts have been made, a message indicating failure is displayed. Success is announced to the user after the ESP32 receives an IP address, as indicated by emission of the `STA_GOT_IP` event.

3.3 Tests

OWL has been extensively tested, both during development and after completion, on multiple levels:

- firmware tests, ensuring application logic is correct;
- electrical tests, ensuring the power supply is functional and all components of the system are connected properly;
- battery life tests, monitoring current consumption and time of operation on a single charge.

The tests have shown that OWL has fully implemented its specification.

3.3.1 Firmware tests

When developing software, it is usually recommended to write unit tests. This enables testing individual system components in isolation, mocking the surrounding layers of abstraction. In microcontroller firmware however, this is often problematic, since the code can be very tightly coupled with the hardware. Writing unit tests for such code requires mocking the hardware, which can potentially be more complex than the firmware itself. What can be unit tested in embedded systems is the application logic, containing processing independent of the hardware. For simple systems however, there may not be much hardware-independent application logic to test. In such cases, it is justified to skip unit tests in favor of performing incremental testing of new code with the hardware, followed by more extensive integration tests. This is the case in OWL – the firmware consists mostly of modules responsible for hardware communication, connected by thin application logic

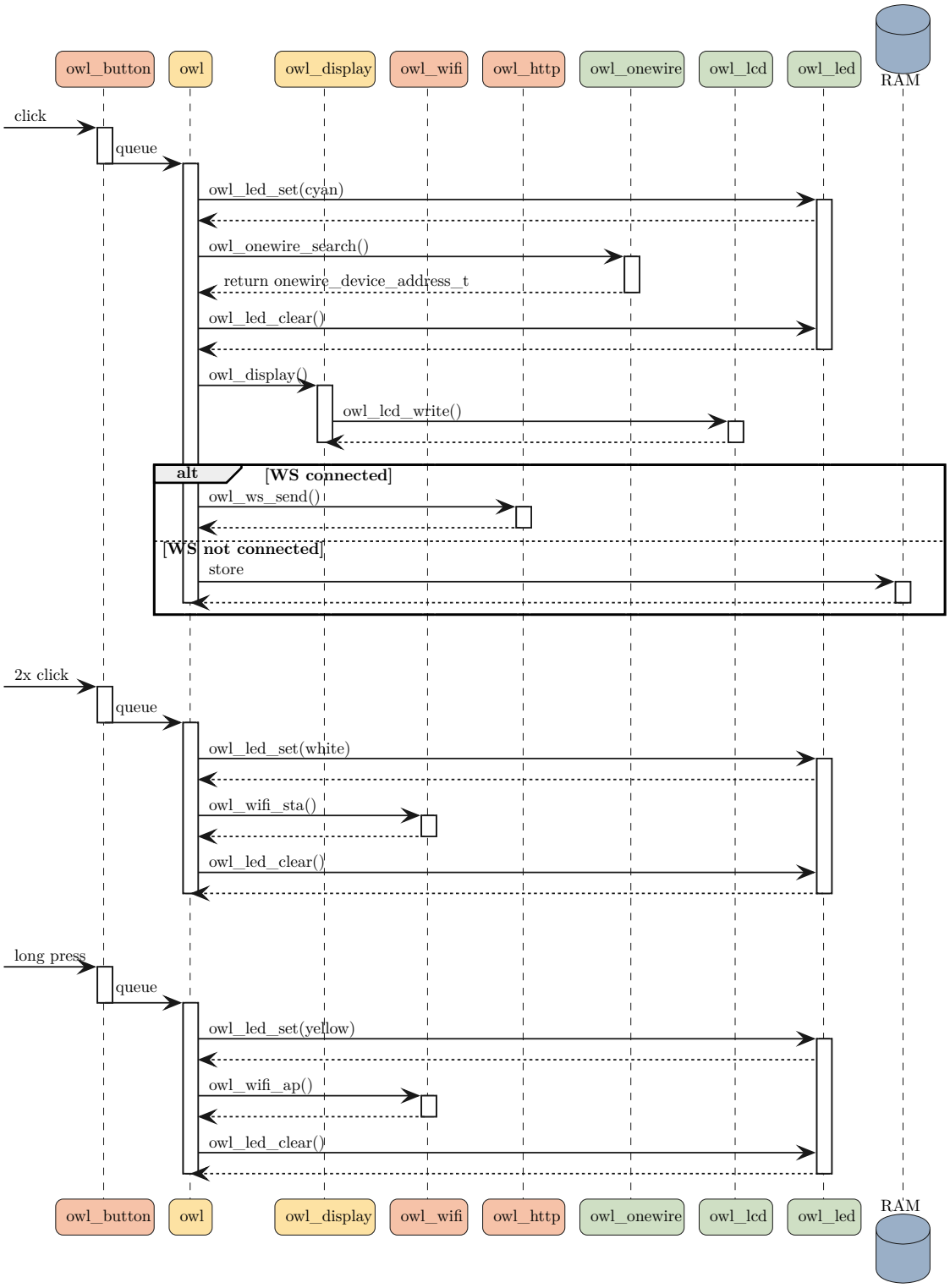


Figure 3.4: Handling of button events. Single click initiates a 1-Wire search and handles the result, depending on the WebSocket status (WS connected/not connected). Double click reinitializes Wi-Fi STA mode. Long press enables Wi-Fi SoftAP mode. Source: own work.

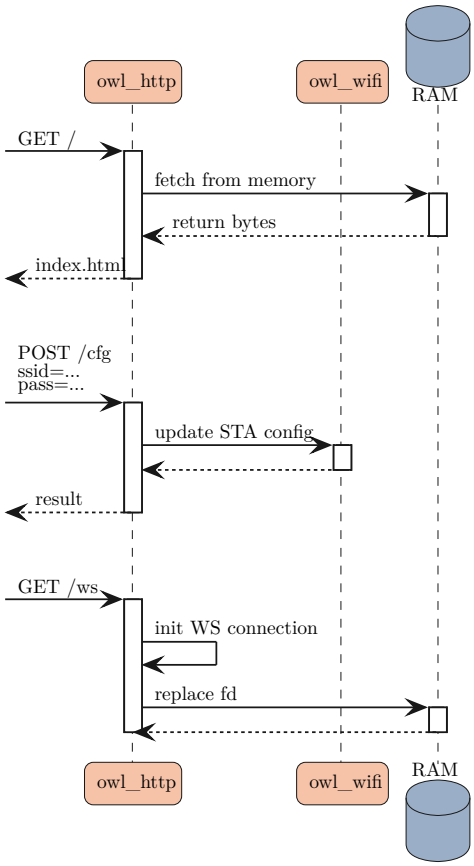


Figure 3.5: Handling of HTTP events (incoming requests). `GET /` fetches `index.html` from RAM and sends the contents. `POST /cfg` replaces the current Wi-Fi credentials with those sent in request path parameters. `GET /ws` initiates a WebSocket connection, whose file descriptor is then stored in RAM. Source: own work.

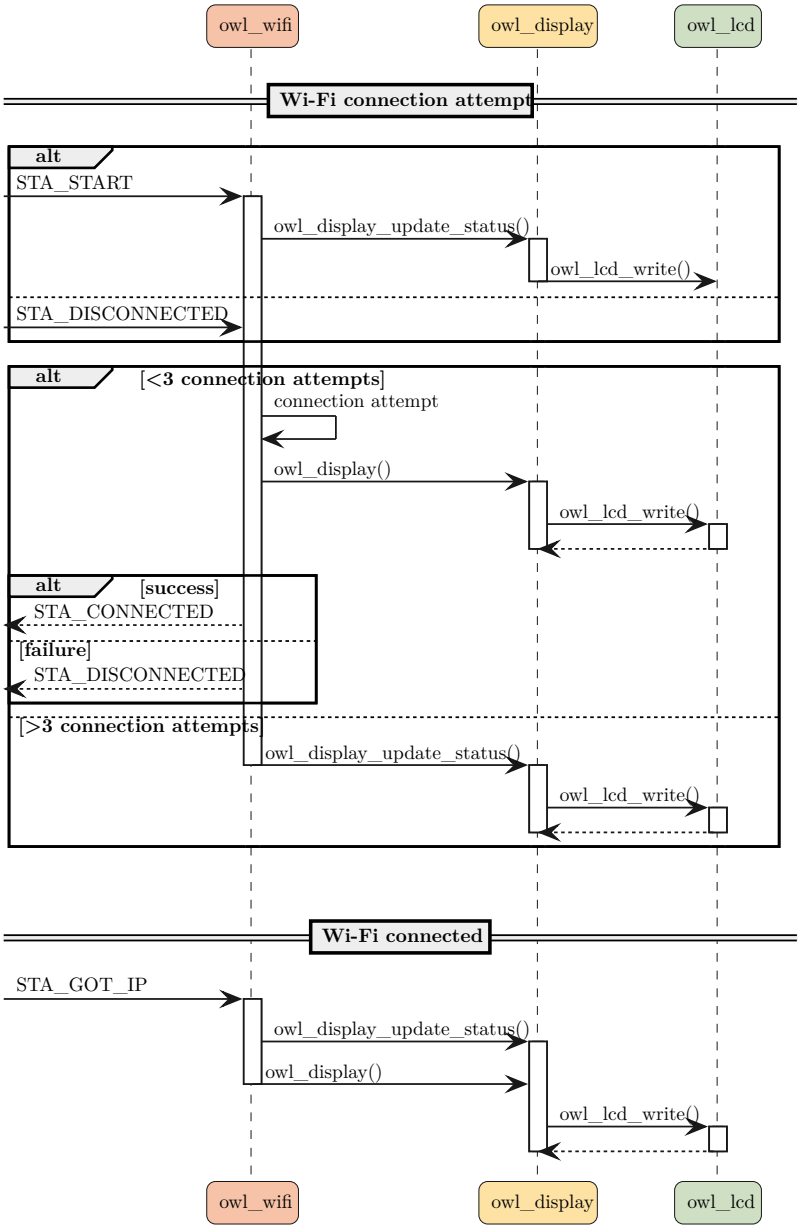


Figure 3.6: Handling of Wi-Fi events. The events are emitted by the IDF component itself. A Wi-Fi connection attempt is made in response to either `STA_START` or `STA_DISCONNECTED` events. Up to three connection attempts are made, each emitting either `STA_CONNECTED` or `STA_DISCONNECTED`, based on the outcome. When connection succeeds, the ESP32 will receive an IP address – this causes the `STA_GOT_IP` event to be emitted, and displays an update. Source: own work.

without much processing. Therefore, no unit tests for OWL have been written and a focus has been placed on incremental and integration tests.

During development, OWL was assembled on a breadboard and powered via the USB-C connector on the ESP32 development board. This simplified the system, allowing performing functional tests without considering potential power supply issues. Numerous test of functionalities have been performed incrementally as the system was being developed.

- Initially a simple application blinking an LED and emitting logs to UART was created, to test the toolchain and basic development board functionality.
- Then, the 1-Wire component from ESP-IDF component registry was used to develop the address readout, which was then tested on a DS18B20 temperature sensor, transmitting the read addresses via UART.
- The button component from ESP-IDF component registry was then added. Multiple button events have been tested with UART log messages, and finally the component has been incorporated to the existing 1-Wire readout logic.
- The Wi-Fi and HTTP server modules were then developed in conjunction, and tested using `curl` and the `index.html` file.
- Lastly, the LCD was connected, at first showing different messages and colors in a loop. After ensuring the communication worked properly, it was then integrated with the rest of the system.

This incremental testing approach helped ensure that all modules are functional when operating independently. This however isn't enough for validating that the system can function as a whole, since the integration can fail or cause unexpected conditions for the modules. Many tests, exploring as much corner cases as possible have been performed. The following 'happy path' normal operation scenarios have been tested, all the time previewing the status page and UART logs:

- reading 1-Wire addresses with no WebSocket connection active;
- configuring the Wi-Fi credentials in SoftAP mode and testing 1-Wire address transmission;
- connecting to a Wi-Fi network in STA mode and testing 1-Wire address transmission again.

The following unexpected or error conditions have been tested, all the time previewing the status page and UART logs:

- reading an empty 1-Wire bus with or without a WebSocket connection;
- performing operations rapidly so as to disturb the display logic;
- generating unexpected button events;
- attempting connection to a nonexistent network/with invalid credentials;
- disconnecting and reconnecting the WebSocket client and performing 1-Wire address reads during the process;
- sending invalid requests to the HTTP server via `curl`;

- disabling the Wi-Fi network that the device is connected to.

This process has helped to find and fix multiple bugs or unwanted behaviors, such as the display being blocked by long-lasting messages, or displayed status pages being inconsistent with current device state. OWL has successfully passed all those tests, it is therefore reasonable to assume that the firmware will fulfil all the requirements.

3.3.2 Electrical tests

Many of the electrical connectivity tests were inherently coupled with the firmware tests – for instance, testing the LCD required connecting it to the I^2C bus. In that case, test have shown that pullup resistors are required for stable operation of the I^2C bus. Connection to the button and a 1-Wire device were also tested along with the firmware. Additionally, Wi-Fi performance in low reception areas was tested, showing performance similar to that of a modern smartphone.

The power system required additional testing, independent of the firmware. The initial version of the power system, assembled on the breadboard, proved rather ineffective – the output voltage was close to the desired 3.3 V, however upon power up of the Wi-Fi module a current spike caused the voltage to drop, triggering a reboot of the ESP32 module. Decoupling capacitors were added, which helped the device to boot successfully once every few tries, which was still not sufficient. Closer analysis of the issue indicated that the problem lay not in the power system itself, but rather in the breadboard – weak connectors caused significant voltage drops on the power lines. When OWL was assembled with proper soldered contacts, the issue was resolved. The device was then tested with a bench power supply and demonstrated the ability to boot up successfully in the full battery voltage range of 3.0 V to 4.2 V. The battery charging was also tested, during a full charge cycle, including operation of the device during charging.

3.3.3 Battery life tests

Measurements have been performed of OWL's current consumption, using a multimeter connected in series with the battery. Due to the bursty nature of Wi-Fi current requirements, it was impossible to achieve precise measurements using this method. The following approximate average values have been measurd:

- 144 mA – STA mode, connecting to Wi-Fi;
- 96 mA – STA mode, no Wi-Fi connection;
- 45 mA – STA mode, no Wi-Fi connection, dimmed backlight;
- 60 mA – STA mode, active Wi-Fi connection;
- 120 mA – SoftAP mode, dimmed backlight.

The backlight at full brightness consumes about 50 mA more current that when dimmed – the firmware has therefore been optimized to dim the backlight when the status page is displayed. The status page is displayed for the most time, and therefore influences the battery life the most, at the same time not requiring the immediate readability provided by backlight's full brightness due to its slow-changing nature.

For stable operation in STA mode, with very occasional bursts of current caused by use of the device, an average current consumption of 61 mA is assumed. Considering the battery's capacity of 1050 mAh, the battery should last for $\frac{1050\,\mathrm{mAh}}{65\,\mathrm{mA}} \approx 17\,\mathrm{h}$. This has

been confirmed by observational tests – the device was turned on at 19:00, used very occasionally for address readout and WebSocket connectivity, and powered off due to the battery discharging shortly after 12:00 the following day, after approximately 17 h. Afterwards, the charging time until a full charge was measured to be just above 2 h. The lifetime is more than enough for operating the device during a full work day, and the charging time is relatively short, which ensures undisturbed use of the device. The battery lifetime of 17 h would be approximately halved if the display backlight was kept at full brightness – the dimming was therefore a very much worthwile optimization. The battery performance has been deemed sufficient considering the use cases of the device.

Conclusion

The thesis describes the successful development of OWL – a portable, hand-held 1-Wire address reader, to be used by engineers working on the ECAL-P detector in the LUXE experiment. All the requirements have been met, with OWL being:

- battery powered, rechargeable and compact;
- capable of reading 1-Wire addresses and communicating them to the user directly or transmitting them (via UART or Wi-Fi);
- usable as-is or with Wi-Fi, connecting to a computer for easier use;
- configurable without requiring dismantling.

The device is based on an ESP32-S3-WROOM-1 module, utilizes a push button and an LCD with RGB backlight as the primary user interface, and a LiPo battery as the power supply. Incremental tests of the firmware have helped produce a reliable product, with integration tests confirming it is fully functional. Its estimated single-charge usage time of 17 h is sufficient for it to be convenient to operate.

OWL in its current form is not a final tool, but rather a fully functional prototype. Its lack of an enclosure is the only serious inconvenience and apart from that, OWL is ready to be used in the LUXE experiment. As usual, the device can still be improved, with potential further developments including:

- design of an enclosure to accommodate the device, for durability and easier handling;
- utilizing a single USB-C connector for charging and log transmission;
- further battery-life optimizations to the firmware, such as reducing clock speed and use of sleep modes;
- handling of multiple WebSocket connections;
- full control of the device via the OWL panel;
- design of a printed circuit board for the device, replacing modules with discrete components.

Bibliography

[1] Wydział Fizyki Uniwersytetu Warszawskiego. (2025) Krok w kierunku wrzenia próżni. [Online]. Available: https://m.fuw.edu.pl/aktualnosci-all/news9496.html

[2] LUXE Collaboration, H. Abramowicz, M. Almanza Soto *et al.*, "Technical design report for the LUXE experiment," *The European Physical Journal Special Topics*, vol. 233, pp. 1709–1974, 2024. [Online]. Available: https://doi.org/10.1140/epjs/s11734-024-01164-9

[3] J. Bogusz, *Lokalne interfejsy szeregowe w systemach cyfrowych*, 1st ed. Warszawa: Wydawnictwo BTC, 2004.

[4] Maxim Integrated, *DS18B20 Programmable Resolution 1-Wire Digital Thermometer*, 2019. [Online]. Available: https://www.analog.com/media/en/technical-documentation/data-sheets/ds18b20.pdf

[5] ——, *What is an iButton Device?*, 2006, application note 3808. [Online]. Available: https://www.analog.com/media/en/technical-documentation/tech-articles/what-is-an-ibutton-device.pdf

[6] Atmel Corporation, *ATmega328P 8-bit AVR Microcontroller with 32K Bytes In-System Programmable Flash*, datasheet 7810D–AVR–01/15. [Online]. Available: https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf

[7] Espressif Systems, *ESP32-S3 Series Datasheet*, 2025. [Online]. Available: https://documentation.espressif.com/esp32-s3_datasheet_en.pdf

[8] University of Wisconsin-Madison. GPIO pins. [Online]. Available: https://ece353.engr.wisc.edu/gpio-pins/gpio-pins/

[9] R. Teja. (2024) Basics of UART communication. [Online]. Available: https://www.electronicshub.org/basics-uart-communication/

[10] Raspberry Pi Ltd, *RP2040 Datasheet*. [Online]. Available: https://pip-assets.raspberrypi.com/categories/814-rp2040/documents/RP-008371-DS-1-rp2040-datasheet.pdf

[11] Espressif Systems, *ESP32-S3 Technical Reference Manual*, 2025. [Online]. Available: https://documentation.espressif.com/esp32-s3_technical_reference_manual_en.pdf

[12] ——. 1-Wire bus component. [Online]. Available: https://components.espressif.com/components/espressif/onewire_bus/versions/1.0.4/readme

[13] A. S. Tanenbaum and D. J. Wetherall, *Computer Networks*, 5th ed. Pearson Prentice Hall, 2011.

[14] Raspberry Pi Ltd, *Raspberry Pi Pico W Datasheet*. [Online]. Available: https://pip-assets.raspberrypi.com/categories/686-raspberry-pi-pico-w/documents/RP-008312-DS-1-pico-w-datasheet.pdf

[15] Espressif Systems, *ESP32-S3-WROOM-1 & WROOM-1U Datasheet*, 2025. [Online]. Available: https://documentation.espressif.com/esp32-s3-wroom-1_wroom-1u_datasheet_en.pdf

[16] A. Dunkels and lwIP Developers. lwip 2.1.x documentation. [Online]. Available: https://www.nongnu.org/lwip/2_1_x/index.html

[17] MDN Web Docs. An overview of HTTP. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Overview

[18] I. Fette and A. Melnikov. (2011) The WebSocket protocol. RFC 6455. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc6455

[19] R. Barry, *Mastering the FreeRTOS Real Time Kernel: A Practical Guide*. Amazon.com, Inc., 2024. [Online]. Available: https://freertos.org/Documentation/02-Kernel/07-Books-and-manual/01-RTOS_book

[20] Espressif Systems. ESP-IDF API guides - build system. [Online]. Available: https://docs.espressif.com/projects/esp-idf/en/v5.5.1/esp32s3/api-guides/build-system.html

[21] K. Hasan, N. Tom, and M. R. Yuce, "Navigating battery choices in IoT: An extensive survey of technologies and their applications," *Batteries*, vol. 9, no. 12, 2023. [Online]. Available: https://www.mdpi.com/2313-0105/9/12/580

[22] UMW, *TP4056 1A Standalone Linear Li-Ion Battery Charger*. [Online]. Available: https://www.umw-ic.com/static/pdf/493f603e988f83ce21e556bca0c516fd.pdf

[23] Renata SA, *Rechargeable lithium-ion polymer batteries guidelines*. [Online]. Available: https://www.renata.com/en/downloadfile/renata-rechargeable-lipo-guidelines/?fileid=f871fff174d88fe997723e20ed

[24] Pololu Corporation. (2025) Pololu 3.3v step-up/step-down voltage regulator s8v9f3. [Online]. Available: https://www.pololu.com/product/4964

[25] [Online]. Available: https://www.waveshare.com/wiki/ESP32-S3-DEV-KIT-N8R8

[26] DFRobot. Gravity: I2C 16x2 Arduino LCD with RGB Font Display. [Online]. Available: https://wiki.dfrobot.com/Gravity___I2C_16x2_Arduino_LCD_with_RGB_Font_Display_SKU___DFR0554

[27] Espressif Systems. Espressif IoT development framework: Examples. [Online]. Available: https://github.com/espressif/esp-idf/tree/release/v5.5/examples

[28] DFRobot. (2023) DFRobot_RGBLCD1602. Arduino Library, Version 2.0.1. [Online]. Available: https://github.com/DFRobot/DFRobot_RGBLCD1602

[29] Espressif Systems. LED strip driver component. [Online]. Available: https://components.espressif.com/components/espressif/led_strip/versions/3.0.2/readme?language=en